Bản đồ project 88Book

Folder nào làm gì, file nào có tác dụng gì, và muốn sửa chức năng thì sửa ở đâu

1. Tổng quan nhanh

88Book là website bán sách fullstack. Frontend hiển thị giao diện, backend cung cấp REST API, database lưu dữ liệu bằng SQL Server. Khi sửa một chức năng, thường cần xem cả 3 lớp: UI trên frontend, API/controller trên backend, và bảng dữ liệu hoặc migration trong database.

- Frontend chạy ở port 5174, source chính nằm trong thư mục src.
- Backend chạy ở port 5000, source chính nằm trong thư mục backend.
- Database dùng SQL Server, script nằm trong thư mục database.
- File cấu hình mật khẩu database nằm ở backend/.env, không nên commit lên git.

2. Bảng folder/file cấp cao

Folder/File	Vai trò	Khi nào cần đụng tới
src/	Source frontend React: UI, page, context, API client.	Khi sửa giao diện, luồng người dùng, cart/auth ở phía browser.
backend/	Source backend Express: API, route, controller, auth middleware, DB connection.	Khi sửa logic nghiệp vụ, route API, phân quyền, xử lý database.
database/	SQL schema, seed data, migration script.	Khi thêm/sửa bảng, cột, quan hệ dữ liệu hoặc dữ liệu mẫu.
package.json	Script và dependency frontend.	Khi thêm thư viện frontend hoặc đổi lệnh build/dev.
backend/package.json	Script và dependency backend.	Khi thêm thư viện backend hoặc đổi lệnh start/dev/migrate.
vite.config.ts	Cấu hình Vite, React plugin, Tailwind plugin, alias.	Khi sửa cấu hình build/dev frontend.
docker-compose.yml	Cấu hình SQL Server và backend khi chạy Docker.	Chỉ dùng nếu máy có Docker và muốn chạy container.
.gitignore	Danh sách file không đưa vào git.	Khi cần ignore env, log, node_modules, dist, build output.
README.md	Tài liệu chính của toàn bộ project.	Khi muốn cập nhật hướng dẫn setup, API, cấu trúc project.

3. Frontend: file nào làm gì

File	Công dụng	Sửa khi muốn thay đổi
src/main.tsx	Entry point, render React app vào DOM.	Hiếm khi sửa; chỉ sửa khi đổi cách mount app/provider toàn cục.
src/app/App.tsx	Component gốc, quản lý page state, load books, điều hướng home/browse/cart/checkout/account/admin.	Sửa flow điều hướng, mapping dữ liệu API sang UI, logic tổng hợp toàn app.
src/api/client.js	Wrapper gọi API: authAPI, booksAPI, cartAPI, ordersAPI, adminAPI, usersAPI.	Sửa endpoint frontend gọi tới backend, thêm API mới, đổi base URL.
src/contexts/AuthContext.jsx	Quản lý login/register/logout, JWT trong	Sửa đăng nhập, đăng ký, logout, kiểm tra

	localStorage, user state.	session/token.
src/contexts/CartContext.jsx	Quản lý cart state và gọi API giỏ hàng.	Sửa thêm/xóa/sửa số lượng giỏ hàng ở frontend.
Navigation.tsx	Navbar, search box, nút cart/account/admin/logout.	Sửa menu, search trên thanh top, hiển thị role user.
BookCard.tsx	Card sách, nút wishlist, add to cart, view detail.	Sửa giao diện từng sách, giá, rating, nút thao tác.
BookDetailsModal.tsx	Modal chi tiết sách.	Sửa nội dung chi tiết, layout modal, nút add to cart trong modal.
BrowsePage.tsx	Trang duyệt sách: search, filter category, sort.	Sửa logic tìm kiếm/lọc/sắp xếp hiển thị frontend.
CartPage.tsx	Trang giỏ hàng.	Sửa layout cart, nút tăng/giảm/xóa, tổng tiền.
CheckoutPage.tsx	Form checkout và gọi tạo đơn hàng.	Sửa luồng đặt hàng, form địa chỉ/thanh toán.
AccountPage.tsx	Login/register/profile/orders/addresses/wish list.	Sửa tài khoản, profile, địa chỉ, wishlist ở giao diện.
AdminDashboard.tsx	Dashboard admin quản lý sách, user, recommendation.	Sửa màn admin, CRUD sách/user/recommendation.
src/styles/theme.css	CSS theme chính.	Sửa màu sắc, spacing, card, animation, layout.

4. Backend: file nào làm gì

File	Công dụng	Sửa khi muốn thay đổi
backend/server.js	Khởi tạo Express app, CORS, JSON parser, health check, mount routes, error handler.	Sửa cấu hình server, CORS, port, middleware chung.
backend/config/database.js	Tạo pool kết nối SQL Server và helper query.	Sửa cách kết nối DB, config mssql, mapping parameter query.
backend/middleware/auth.js	Xác thực JWT và kiểm tra role.	Sửa phân quyền, token expired, logic admin/buyer.
routes/auth.js	Định nghĩa endpoint auth.	Thêm/sửa route login/register/me/admin.
routes/books.js	Định nghĩa endpoint sách.	Thêm/sửa route books/recommendations/CRUD.
routes/cart.js	Định nghĩa endpoint cart.	Thêm/sửa route giỏ hàng.
routes/orders.js	Định nghĩa endpoint orders.	Thêm/sửa route tạo đơn, hủy đơn, admin duyệt đơn.
routes/admin.js	Định nghĩa endpoint admin dashboard.	Thêm/sửa route quản trị user/book/order/stats.
routes/users.js	Định nghĩa endpoint profile/address/wishlist.	Thêm/sửa route user cá nhân.
authController.js	Logic đăng ký, đăng nhập, tạo JWT, tạo admin.	Sửa rule email/password, payload token, response auth.
bookController.js	Logic lấy sách, search/filter/sort, recommendation, CRUD sách.	Sửa nghiệp vụ sách, query list sách, quyền chỉnh sách.
cartController.js	Logic thêm/sửa/xóa giỏ hàng.	Sửa kiểm tra tồn kho khi thêm cart, response cart.
orderController.js	Logic tạo đơn, order items, trừ kho, hủy/duyệt/cập nhật trạng thái.	Sửa checkout, trạng thái đơn hàng, khôi phục tồn kho.
adminController.js	Logic quản trị user, books, recommendations, orders, stats.	Sửa dashboard admin và thống kê.
userController.js	Logic profile, đổi mật khẩu, địa chỉ, wishlist.	Sửa thông tin user, address book, wishlist.

5. Database: file/bảng nào làm gì

File/Bảng	Công dụng	Sửa khi muốn thay đổi
database/init.sql	Script SQL tổng hợp tạo bảng và seed data.	Khi muốn chạy trực tiếp trong SSMS hoặc cập nhật seed/schema tổng hợp.
database/migrations/run.js	Migration chính tạo các bảng app cần.	Khi thêm bảng/cột chính hoặc sửa schema bằng Node migration.
add-rating-column.js	Migration bổ sung rating cho books.	Khi thay đổi logic rating sách.
add_wishlist_address.js	Migration tạo user_addresses và wishlists.	Khi sửa địa chỉ giao hàng hoặc wishlist.
users	Lưu tài khoản, mật khẩu hash, role.	Khi sửa đăng nhập, profile, phân quyền.
books	Lưu sách, giá, category, tồn kho, ảnh, rating.	Khi sửa catalog sách, sản phẩm, tồn kho.
cart_items	Lưu giỏ hàng theo user.	Khi sửa giỏ hàng.
orders	Lưu đơn hàng tổng.	Khi sửa checkout, trạng thái đơn, thanh toán/giao hàng.
order_items	Lưu từng dòng sách trong đơn.	Khi sửa chi tiết đơn hoặc giá chốt tại thời điểm đặt.
user_addresses	Lưu địa chỉ giao hàng.	Khi sửa address book.
wishlists	Lưu sách yêu thích của user.	Khi sửa wishlist.
featured_recommendations	Lưu sách được admin chọn làm recommendation.	Khi sửa mục gợi ý trên trang chủ.

6. Muốn sửa chức năng thì sửa ở đâu

Chức năng	Frontend cần xem	Backend cần xem	Database cần xem
Đăng nhập/đăng ký	AuthContext.jsx, AccountPage.tsx, api/client.js authAPI	routes/auth.js, authController.js, middleware/auth.js	users
Đăng xuất / lưu token	AuthContext.jsx, Navigation.tsx	middleware/auth.js nếu đổi JWT rule	Không cần, trừ khi lưu session server-side.
Danh sách sách	App.tsx, BrowsePage.tsx, BookCard.tsx, api/client.js booksAPI	routes/books.js, bookController.js	books
Tìm kiếm/lọc/sắp xếp sách	BrowsePage.tsx, api/client.js booksAPI.getAll	bookController.js getAllBooks	books: title, author, category, price, rating
Chi tiết sách	BookDetailsModal.tsx, BookCard.tsx	routes/books.js, bookController.js getBookById	books
Thêm sách vào giỏ	BookCard.tsx, BookDetailsModal.tsx, CartContext.jsx	routes/cart.js, cartController.js addToCart	cart_items, books
Sửa số lượng giỏ hàng	CartPage.tsx, CartContext.jsx	cartController.js updateCartItem	cart_items, books
Checkout / tạo đơn	CheckoutPage.tsx, App.tsx ordersAPI.create	routes/orders.js, orderController.js createOrder	orders, order_items, cart_items, books
Hủy đơn	AccountPage.tsx nếu thêm UI hủy đơn	orderController.js cancelOrder	orders, order_items, books
Admin thêm/sửa/xóa sách	AdminDashboard.tsx, api/client.js booksAPI/adminAPI	bookController.js create/update/deleteBook, routes/books.js	books
Admin quản lý user	AdminDashboard.tsx	adminController.js, routes/admin.js	users

Admin thống kê	AdminDashboard.tsx	adminController.js getAdminStats	users, books, orders, order_items
Recommendation trang chủ	App.tsx, AdminDashboard.tsx	bookController.js, adminController.js	featured_recommendations, books
Profile user	AccountPage.tsx, App.tsx, api/client.js usersAPI	userController.js updateUserProfile	users
Đổi mật khẩu	AccountPage.tsx nếu có form, api/client.js usersAPI.changePassword	routes/users.js, userController.js changePassword	users.password
Địa chỉ giao hàng	AccountPage.tsx, App.tsx, addressesAPI	userController.js address methods, routes/users.js	user_addresses
Wishlist	BookCard.tsx, App.tsx, AccountPage.tsx, wishlistAPI	userController.js wishlist methods, routes/users.js	wishlists, books
Cấu hình API URL	src/api/client.js, .env hoặc .env.local	backend/server.js CORS FRONTEND_URL	Không cần.
Cấu hình database	Không cần.	backend/.env, config/database.js	SQL Server, DB book_store

7. Checklist trước khi sửa code

Việc cần làm	Lý do
Xác định chức năng thuộc frontend, backend hay database.	Tránh sửa nhầm nơi và tạo lỗi lan sang module khác.
Tìm API đang được frontend gọi trong src/api/client.js.	Biết endpoint và payload frontend gửi lên backend.
Tìm route tương ứng trong backend/routes.	Biết request đi vào controller nào.
Đọc controller trước khi sửa query.	Hiểu nghiệp vụ hiện tại, response format và lỗi có thể trả về.
Nếu thay schema, thêm migration trong database/migrations.	Giữ database của người khác có thể nâng cấp được.
Chạy npm run build sau khi sửa frontend.	Bắt lỗi import, JSX, build CSS/JS.
Chạy node --check cho backend/database sau khi sửa backend.	Bắt lỗi cú pháp JavaScript.
Không commit .env và node_modules.	Tránh lộ mật khẩu và làm repo nặng.

8. Ghi chú vận hành

- Frontend chạy bằng: npm run dev ở thư mục gốc project.
- Backend chạy bằng: npm run dev trong thư mục backend.
- Migration chạy bằng: npm run migrate trong thư mục backend.
- Nếu backend báo lỗi database, kiểm tra SQL Server, port 1433, DB_USER, DB_PASSWORD trong backend/.env.
- Nếu frontend không gọi được API, kiểm tra backend đã chạy chưa và VITE_API_URL có đúng http://localhost:5000/api không.

9. Công nghệ sử dụng và cách chạy

Phần này ghi rõ từng lớp của project 88Book đang chạy bằng công nghệ gì, chạy ở đâu, file cấu hình nào quyết định port/kết nối, và lệnh nào dùng khi phát triển local.

Lớp	Công nghệ chính	Port / URL local	File quyết định
Frontend	React 18, Vite 6, TypeScript/TSX, Tailwind CSS 4, Radix UI, lucide-react	http://localhost:5174	package.json, vite.config.ts, .env, src/main.tsx, src/api/client.js
Backend	Node.js 18, Express 4, ES Modules, JWT, bcryptjs, CORS, dotenv, mssql	http://localhost:5000/api	backend/package.json, backend/server.js, backend/config/database.js, backend/Dockerfile
Database	Microsoft SQL Server 2022, database book_store, schema dbo, Docker volume sqlserver_data	localhost:1433 hoặc service sqlserver trong Docker	docker-compose.yml, database/init.sql, database/migrations/*.js

9.1 Frontend chạy bằng gì

Mục	Chi tiết
Framework/thư viện UI	React 18.3.1, render từ src/main.tsx vào #root trong index.html. Component gốc là src/app/App.tsx.
Build tool/dev server	Vite 6.4.2. Lệnh dev trong package.json là npm run dev, thực chất chạy vite --port 5174.
Ngôn ngữ/source	TypeScript/TSX cho app chính, có một số file .jsx cho context như AuthContext.jsx và CartContext.jsx.
CSS/UI	Tailwind CSS 4 qua @tailwindcss/vite, CSS nằm trong src/styles/*.css; component UI nằm trong src/app/components/ui.
API base URL	Frontend gọi backend qua VITE_API_URL trong file .env, hiện là http://localhost:5000/api. Nếu không có biến này, src/api/client.js fallback về http://localhost:5000/api.
Lệnh thường dùng	Ở thư mục gốc project: npm install nếu chưa có node_modules; npm run dev để chạy frontend; npm run build để build production ra dist/.

9.2 Backend chạy bằng gì

Mục	Chi tiết
Runtime	Node.js, backend dùng ES Modules vì backend/package.json có "type": "module".
Framework API	Express 4.22.1. File backend/server.js khởi tạo app, middleware JSON/urlencoded, CORS, health check và mount các route.
Port/API root	PORT mặc định là 5000. Health check: GET http://localhost:5000/api/health. Các endpoint chính nằm dưới /api/auth, /api/users, /api/books, /api/cart, /api/orders,

	/api/admin.
Bảo mật/xác thực	jsonwebtoken dùng cho JWT, bcryptjs dùng hash mật khẩu, middleware xác thực nằm ở backend/middleware/auth.js.
Kết nối database	Driver mssql 11.0.1, cấu hình pool nằm ở backend/config/database.js và đọc DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_ENCRYPT, DB_TRUST_SERVER_CERTIFICATE từ env.
Lệnh thường dùng	Trong thư mục backend: npm install nếu chưa có node_modules; npm run dev để chạy nodemon server.js; npm start để chạy node server.js; npm run migrate để chạy database/migrations/run.js.
Docker backend	backend/Dockerfile dùng node:18-alpine, expose port 5000, command mặc định npm run dev. docker-compose build service backend từ thư mục ./backend.

9.3 Database chạy bằng gì

Mục	Chi tiết
Hệ quản trị CSDL	Microsoft SQL Server 2022. docker-compose.yml dùng image mcr.microsoft.com/mssql/server:2022-latest.
Database chính	Tên database là book_store. docker-compose có service sqlserver-init tạo database nếu chưa tồn tại.
Cổng/kết nối	SQL Server expose 1433:1433. Khi backend chạy trong Docker, DB_HOST=sqlserver; khi chạy local ngoài Docker thường dùng DB_HOST=localhost.
User/mật khẩu dev	docker-compose.yml cấu hình DB_USER=sa và DB_PASSWORD=YourStrong@Passw0rd cho môi trường local/dev. Không dùng mật khẩu này cho production.
Schema/tables	database/init.sql tạo các bảng users, books, cart_items, orders, order_items, user_addresses, wishlists, recommendations và seed dữ liệu mẫu.
Migration	database/migrations/run.js là migration chính; các file add_wishlist_address.js và add-rating-column.js bổ sung cột/bảng theo từng thay đổi.
Dữ liệu Docker	Volume sqlserver_data giữ dữ liệu SQL Server để container restart không mất database.

9.4 Cách chạy local theo từng lớp

1. Chạy database bằng Docker: docker compose up -d sqlserver sqlserver-init. Chờ healthcheck SQL Server báo healthy.
2. Chạy backend: cd backend, npm install nếu cần, sau đó npm run dev. Backend lắng nghe ở http://localhost:5000 và API root là /api.
3. Chạy frontend: ở thư mục gốc project, npm install nếu cần, sau đó npm run dev. Frontend mở ở http://localhost:5174.
4. Nếu muốn chạy cả database và backend bằng Docker, dùng docker compose up --build. Frontend vẫn có thể chạy riêng bằng npm run dev ở thư mục gốc.

5. Kiểm tra nhanh: mở http://localhost:5000/api/health để xác nhận backend chạy; mở http://localhost:5174 để kiểm tra giao diện.

9.5 Luồng kết nối giữa 3 lớp

Từ	Đến	Cách kết nối
Browser/frontend	Backend Express	src/api/client.js dùng fetch đến VITE_API_URL=http://localhost:5000/api, gửi/nhận JSON và đính kèm Authorization: Bearer <token> nếu đã login.
Backend Express	SQL Server	backend/config/database.js tạo ConnectionPool bằng driver mssql, đổi placeholder $1 thành @p1 trước khi query.
Docker backend	Docker SQL Server	Trong docker-compose, backend dùng DB_HOST=sqlserver để gọi service SQL Server cùng network book_store_network.

9.6 File cần sửa khi đổi công nghệ/cấu hình chạy

Nếu đổi port frontend, sửa vite.config.ts và đồng bộ FRONTEND_URL ở backend/.env hoặc docker-compose.yml. Nếu đổi port/backend URL, sửa PORT của backend và VITE_API_URL trong .env. Nếu đổi thông tin SQL Server, sửa DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASSWORD trong môi trường backend và bảo đảm database/init.sql hoặc migration vẫn khớp schema thực tế.